

A minimalist line-art illustration in the background. On the right, a person with glasses and a slight smile is shown from the chest up, holding a large folder or book. The folder is open, and several sheets of paper are visible. In the upper left, there are three diamond shapes of varying sizes. A large, thin arc curves across the top of the page.

Construção de componentes

Você vai aprender como transformar requisitos em especificações claras para programadores, compreendendo sua importância na arquitetura de sistemas e garantindo a construção de componentes padronizados, independentes de tecnologias específicas.

Prof. Marcos Alexandre Fernandes Ferronato

Propósito

A capacidade de transformar requisitos em especificações claras e tecnicamente viáveis é essencial para profissionais de desenvolvimento de sistemas, pois garante a construção de componentes coerentes com a arquitetura definida, promovendo soluções eficazes e aderentes às necessidades dos usuários e às boas práticas do setor.

Objetivos

- Diferenciar os elementos envolvidos na especificação de componentes e sua contribuição para a arquitetura de sistemas eficaz.
- Avaliar os elementos e práticas da construção de componentes como parte essencial da metodologia de desenvolvimento apresentada.
- Relacionar a construção de componentes aos demais processos metodológicos, evidenciando seu impacto no sucesso do projeto.

Introdução

No desenvolvimento de sistemas, um dos desafios mais críticos está em transformar os requisitos levantados junto às partes interessadas em especificações técnicas claras e precisas, que possam ser corretamente interpretadas e implementadas pelos programadores. Isso ocorre porque, muitas vezes, as informações fornecidas pelos stakeholders não seguem as normas ou padrões exigidos para o desenvolvimento de software. Nesse contexto, a especificação de componentes desempenha um papel fundamental, funcionando como uma ponte entre a fase de levantamento de requisitos e a construção efetiva do sistema.

Este conteúdo apresenta um conjunto de ações essenciais para garantir que a especificação represente, de forma consistente e concreta, todas as informações levantadas. Também explora o conceito de especificação como a tradução dos requisitos em elementos de programação, destacando sua importância para a arquitetura de sistemas e para a obtenção de resultados alinhados com as expectativas dos usuários.

A abordagem aqui apresentada propõe uma nova forma de compreender a especificação de componentes, conectando esse processo aos resultados a serem alcançados com a solução. Além disso, será detalhado o último estágio da metodologia estudada: a construção de componentes. Essa etapa visa transformar, com qualidade e dentro dos padrões estabelecidos, os elementos definidos pelo arquiteto de sistemas em componentes funcionais e adequados ao problema identificado.

Ao longo do conteúdo, serão abordadas práticas para garantir que as especificações estejam o mais independentes possível das tecnologias utilizadas, além de discutir os limites de algumas técnicas de especificação, que exigem interpretação devido à variedade de tecnologias disponíveis. Dessa forma, reforça-se a relevância da especificação e construção de componentes como etapas centrais para o sucesso de qualquer projeto de desenvolvimento de sistemas.

Definição de especificação de componentes

Para entendermos os conceitos de especificação de componentes, é necessário compreender quais os elementos estão relacionados a um componente.

Vamos começar conhecendo as características de um componente:

- Tem uma especificação.
- Pode ser distribuído.
- Pode ser empacotado em módulos.
- Deve ser aderente a padrões.
- Deve ser ade
- Tem uma implementação que deve ser baseada em todos os elementos anteriores.

A imagem a seguir representa esse conceito de forma simplificada, para facilitar o entendimento das relações entre a especificação de um componente e seus elementos:



Tipos de componentes

Os componentes mais importantes de um sistema a ser desenvolvido, baseado nos conceitos de arquitetura de sistemas, são:

- Objetos de sistemas prontos.
- SGBD.
- Elementos de comunicação.
- Componentes a serem desenvolvidos para o sistema.

Aqui vamos nos ater ao último tipo.

Especificação de componentes a serem desenvolvidos

Como vimos anteriormente nesta aula, especificar um componente pode ser definido como o processo de traduzir as necessidades das partes interessadas em uma linguagem que os desenvolvedores entendam e consigam implementar.

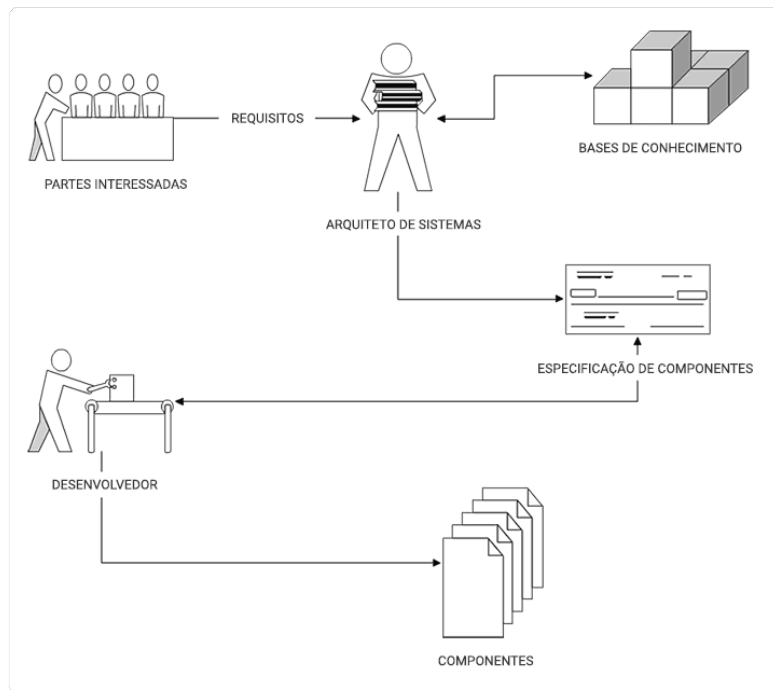
Durante esse processo, é necessário que o arquiteto de sistemas complete as informações técnicas necessárias que as partes interessadas não conseguem informar, baseado em seu framework de conhecimentos.

Esse processo pode ser melhor entendido quando observamos a imagem ao lado.



Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Atenção

Note que todo esse processo tem como objetivo a geração de artefatos de sistemas (componentes) mais assertivos e comprometidos com o resultado final do sistema.

Metodologia/padrões

O arquiteto de sistemas deve se preocupar com os seguintes conceitos quando for especificar um componente.

Primeiro, deve definir as informações quanto ao escopo do componente. Nesse caso, separados em dois elementos (padrões de classes e padrões de objetos).

Depois, deve definir as informações com relação ao propósito, que, no nosso caso, serão criação, estrutura e comportamento.

O esquema a seguir representa bem esse conceito e ajuda no entendimento da especificação de componentes.

		Propósito		
		Criação	Estrutura	Comportamento
Escopo	Classe	Factory Method	Class Adapter	Interpreter

		Propósito		
		Criação	Estrutura	Comportamento
	Objeto			Template Method
		Abstract Method	Object Adapter	Chain of Responsibility
		Builder	Bridge	Command
		Prototype	Composite	Iterator
		Singleton	Decorator	Mediator
			Facede	Observer
			Flyweight	State
			Proxy	Strategy
				Visitor

Marcos Alexandre Fernandes Ferronato.



Atenção

Note que o objetivo aqui não é abordar detalhadamente cada um dos padrões, mas apresentar a você, futuro arquiteto de sistemas, quais são as diretrizes a serem seguidas quando for desenvolver sistemas utilizando o conceito de componentes da arquitetura de sistemas.

Empacotamento de componentes

Uma das mais úteis ferramentas do desenvolvimento de sistemas por componentes é o **empacotamento**.

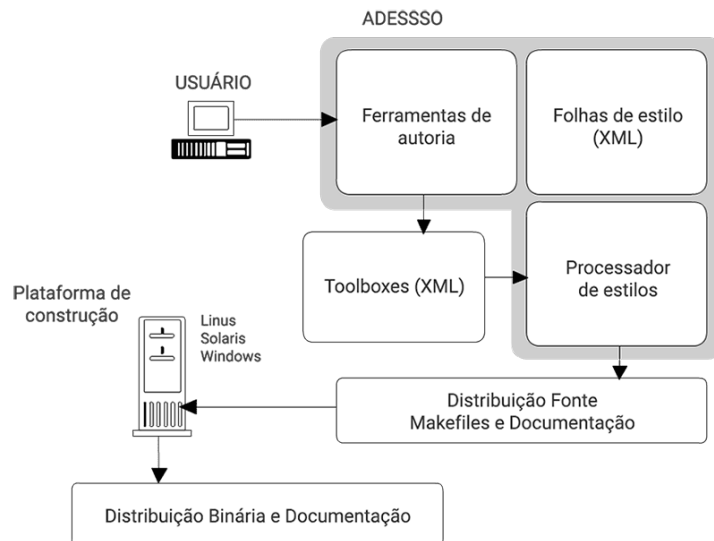
Com esse recurso, o arquiteto consegue resolver o sistema, abstraindo conceitos de mais alto nível, sem se preocupar muito com os conteúdos dos objetos, mas, sim, com sua funcionalidade.

Nesse caso especificamente, os componentes são reunidos por funcionalidade em pacotes que auxiliam muito o arquiteto em sua tarefa de resolver os problemas das partes interessadas.



Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Distribuição de componentes

O arquiteto de sistemas deve ter a preocupação de definir e construir middlewares que conectem as diversas camadas de componentes e, consequentemente, os componentes, de maneira a conseguir uma forte acoplagem para a organização.

Os componentes devem ser distribuídos em containers seguindo as políticas estabelecidas pelos middlewares, ou seja, dentro de um mesmo container somente podem existir componentes que atendam a mesma configuração.

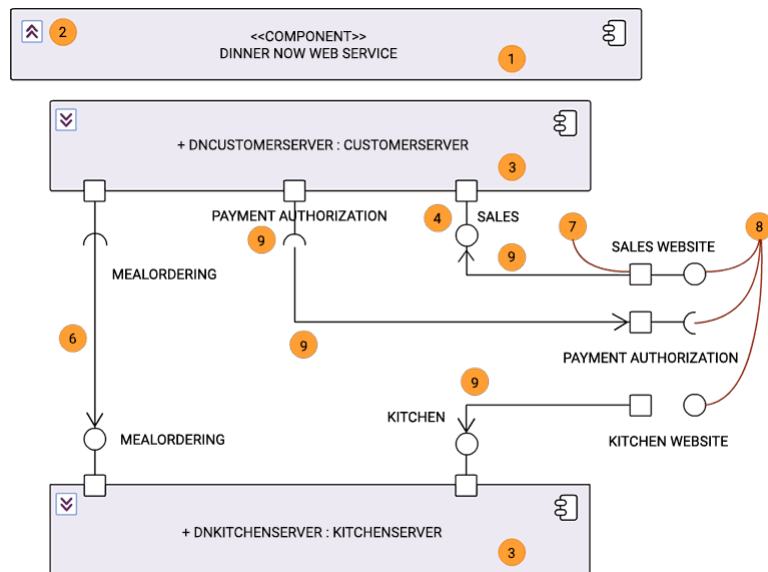
Essa configuração está presente nos descritores de distribuição que, geralmente, são escritos em XML e devem conter informações suficientes para que:

- Os componentes sejam corretamente conectados.
- As camadas contenham os níveis corretos de suporte de configuração.
- Os níveis de segurança corretos para a aplicação estejam implementados.



Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Implementação de componentes

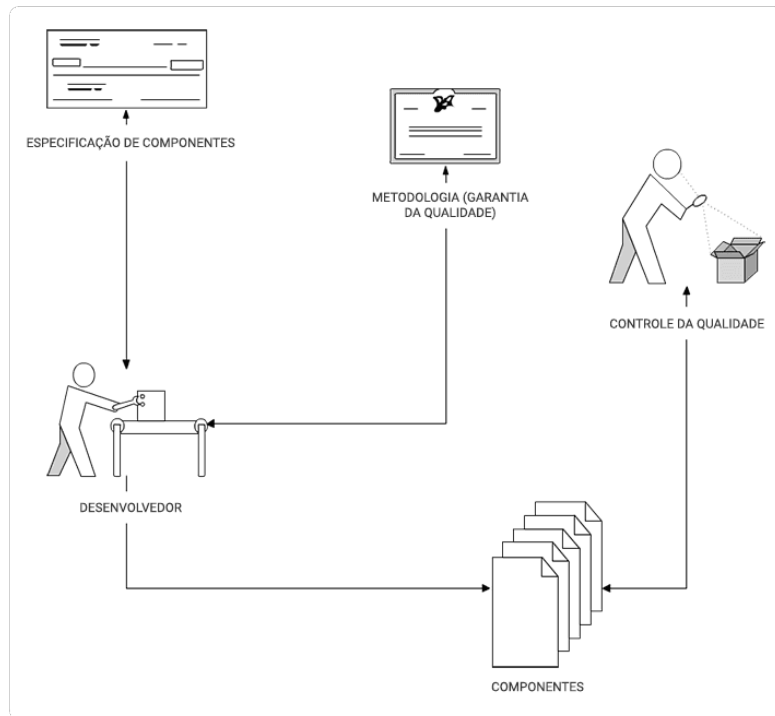
A implementação de componentes é tarefa dos programadores, mas o arquiteto de sistemas também tem sua parcela de responsabilidade neste contexto, gerando especificações que atendam às necessidades e sigam os padrões definidos pela organização.

Outro conjunto de participantes nesse contexto é a área de garantia e controle da qualidade, que tem a função de prover elementos que garantam a qualidade dos componentes produzidos.



Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Verificando o aprendizado

Questão 1

No tocante ao desenvolvimento de software orientado ao reuso, embora o estágio inicial de especificação de requisitos e o estágio de validação sejam comparáveis com outros processos, os estágios intermediários em um processo orientado ao reuso são diferentes.

Nesse caso, são processos em estágios intermediários:

- A Projeto de interface, análise de componentes, projeto arquitetural e testes de aceitação.
- B Análise de componentes, modificação de requisitos, projeto de sistemas com reuso e desenvolvimento e integração.
- C Desenvolvimento de protótipo, projeto arquitetural, análise de componentes e plano de teste.
- D Especificação de sistema, avaliação de mudanças, análise de componentes e desenvolvimento e integração.



A alternativa B está correta.

Os estágios intermediários no desenvolvimento orientado ao reuso incluem análise de componentes, modificação de requisitos, projeto com reuso e desenvolvimento com integração.

Questão 2

Na arquitetura de sistemas, determinado conceito permite que, entre dois elementos de software A e B, seja possível postular alguma mudança de A, que pediria que B fosse mudado (ou, no mínimo, cuidadosamente verificado), a fim de preservar a exatidão global e também postular alguma mudança, que pediria que tanto A como B mudassem juntos para preservar a exatidão global. Isso trata-se do conceito de:

☐ A Polimorfismo.

☒ B Congeneridade.

☐ C Mutabilidade.

☐ D Polidependência.



A alternativa B está correta.

O conceito de congeneridade descreve a necessidade de mudanças coordenadas entre componentes para manter a exatidão global do sistema.

Componentes

Um dos conceitos mestres na arquitetura de sistemas baseada em componentes é que um componente é definido e construído para fornecer um certo nível de serviço.

Como vimos, teremos os componentes separados em função dos grupos de serviços que eles oferecem. Teremos componentes de hardware, de sistema operacional, de SGDB, de aplicativos comerciais e de sistemas da própria corporação já desenvolvidos.

O arquiteto de sistemas, baseado nos requisitos do novo sistema, vai executar o design da nova aplicação, identificando todos os componentes necessários e aplicando reuso aos componentes que já existirem. Somente serão construídos os componentes que não existirem.

É importante que o arquiteto de sistemas conduza a definição e a construção desses novos componentes, de maneira que eles passem a fazer parte do conjunto de componentes a serem reutilizados no futuro.

Componentes

Que compõe, ou entra na composição de alguma coisa; que contribui para formar; constituinte.



Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Ambiente de componentes

Um ambiente componente é um meio ambiente de objetos distribuídos.

Os componentes devem estar em conformidade a um conjunto de regras padrão para que se possa operar nesse ambiente, e um conjunto de serviços de infraestrutura (suporte a transações, segurança, concorrência e assim por diante), na qual o componente de aplicação pode depender.



Atenção

Além disso, vamos nos limitar a esses ambientes de componentes que fornecem os serviços de infraestrutura declarativa, usando uma abordagem por framework (por vezes chamado de “programação baseada em atributo”), em vez de usarmos uma invocação explícita dentro da própria lógica da aplicação.

Fazemos isso pois os frameworks são os mais indicados para fornecer uma base mais sólida para a próxima geração de aplicativos baseados em componentes distribuídos, em escala empresarial.

Nesse contexto, temos as seguintes opções:

	Dependência de Plataforma	
Microsoft COM+	Windows	Nenhuma
Enterprise JavaBeans (EJB)	Nenhuma	Java

Marcos Alexandre Fernandes Ferronato.

Nesta conteúdo, não vamos fazer análise de qual ambiente é melhor e em que condições, tampouco vamos direcionar qual dos dois devem ou não ser usados. Aqui vamos nos limitar a relatar os conceitos de uso dos frameworks sem a preocupação de detalhar particularidades de cada um deles.

Framework CCM – CORBA Component Model

O CCM é um framework de componentes do lado do servidor, cuja finalidade é facilitar o desenvolvimento e a instalação de aplicações distribuídas que utilizam a arquitetura de sistemas por componentes.

Esses componentes podem ser de diversos tipos, como vimos nos conteúdos anteriores.

- Níveis de componentes do CCM

O CCM é dividido em dois níveis de componentes:

Nível básico

Provê uma forma simplificada de distribuir um objeto CORBA como componente.

Nível estendido

Provê um conjunto maior de ações, como as portas de comunicação que representam os elementos de conexão entre os componentes.

- Cinco tipos de modelos do CCM

O CCM é também estruturado em cinco tipos de modelos:

Modelo abstrato

Define os atributos, portas de comunicação e home dos componentes.

Modelo de programação

Composto pela CIDL (Component Implementation Definition Language) e pelo CIF (Component Implementation Framework).

Modelo de Empacotamento

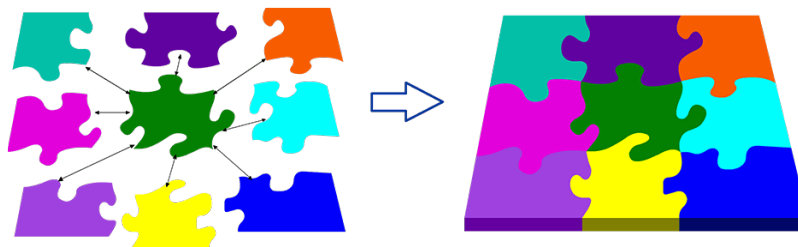
Especifica como os componentes e suas implementações devem ser empacotados.

Modelo de Instalação

Define um mecanismo padrão para a instalação de aplicações.

Modelo de Execução

Define o ambiente de execução para as instâncias do componente.



Arquivos CIDL

Os seguintes elementos são especificados em um arquivo CIDL, no contexto do desenvolvimento na arquitetura de sistemas por componentes:

	Corba usage model	Tipo de api do container	Interface base chaveada	Exemplo
	Stateless	Session	Não	Componentes do Sistem
	Conversational	Session	Nio	Interfaces de Interação
	Durivel	Entity	Nio	Regras de Negdocio

Os componentes são empacotados em DLLs e executados em servidores de componentes.

As implementações dos componentes dependem dos conceitos da programação orientada a aspectos para encaminhar requisições de clientes para os elementos de servidor.

Os componentes não precisam saber como tratar problemas, como a criação de hierarquia de POAs, e localizar serviços do CCM.



Funcionalidades

Para isso, foram definidos os containers, com as seguintes funcionalidades:

- Ativação/desativação de implementações de componentes, preservando recursos (como memória).
- Fornecimento de camada de adaptação com os serviços de transação, persistência, segurança e notificação.
- Fornecimento de camada de adaptação para call-backs.
- Gerenciamento de políticas do POA.

Gerenciamento do ciclo de vida

O gerenciamento do ciclo de vida dos componentes de servidor é feito através de políticas que controlam o momento de ativação/desativação dos componentes:

Method

Ativação/desativação a cada chamada de método, limitando o uso de memória ao tempo de duração da operação, mas acrescentando o custo de ativação e desativação do componente.

Transaction

Ativação/desativação a cada transação. Memória permanece alocada durante a transação.

Component

O container ativa o componente, quando for feita a primeira chamada a alguma de suas operações, e desativa, quando explicitamente requisitado pela aplicação, deslocando a memória utilizada pelo componente.

Container

O componente será ativado quando for feita a primeira chamada a alguma de suas operações e, ao final da execução da mesma, será desativado. Entretanto, a memória permanecerá alocada até que o container decida deslocá-la.

Empacotamentos e distribuição

Em sistemas distribuídos, componentes podem ser implantados em diversos servidores e sistemas operacionais.

Além disso, um componente pode depender de outros componentes, tornando o processo de empacotamento e distribuição bem mais complicado do que se imagina.

CCM descreve componentes e suas dependências usando Open Software Description (OSD), que é um XML Document Type Definition (DTD) definido pelo consórcio www.omg.org.



Componentes são empacotados em DLLs. Package descriptors são documentos XML em conformidade com o OSD e DTD, descrevendo o conteúdo da DLL e suas dependências.

CCM e OSD também definem component assembly descriptors, que descrevem instruções de implantação e topologia dos componentes, e têm como objetivo o suporte à implantação automática dos componentes.

Verificando o aprendizado

Questão 1

Na construção dos componentes de um domínio do problema, há três passos principais. Assinale a alternativa que corresponde à ordem correta de criação desses passos:

☐ A Instalar componentes, gerar código dos componentes e modelar componentes.

☐ B Gerar código dos componentes, modelar componentes e instalar componentes.

☒ C Modelar componentes, gerar código dos componentes e instalar componentes.

☐ D Modelar componentes, instalar componentes e gerar código dos componentes.



A alternativa C está correta.

A construção dos componentes segue a ordem: modelar componentes, gerar seu código e, por fim, instalá-los.

Questão 2

A etapa “modelar componentes” é dividida em três passos: definir domínio do problema, especificar componentes e projetar componentes. Assinale a alternativa que não corresponde à especificação de cada um dos passos dessa etapa:

A No passo “definir domínio do problema”, obtêm-se as especificações UML do domínio do problema a partir estudo do conhecimento do domínio do problema.

B No passo “especificar componentes”, obtêm-se os componentes especificados usando técnicas de DBC.

C No passo “projetar componentes”, faz-se o projeto interno dos componentes, usando técnicas de DBC do método Catalysis [D’Souza, 1999].

D No passo “especificar componentes”, as atividades são realizadas no nível de abstração do projeto interno de componentes.



A alternativa D está correta.

A especificação de componentes ocorre em um nível mais abstrato, e não no nível de projeto interno.

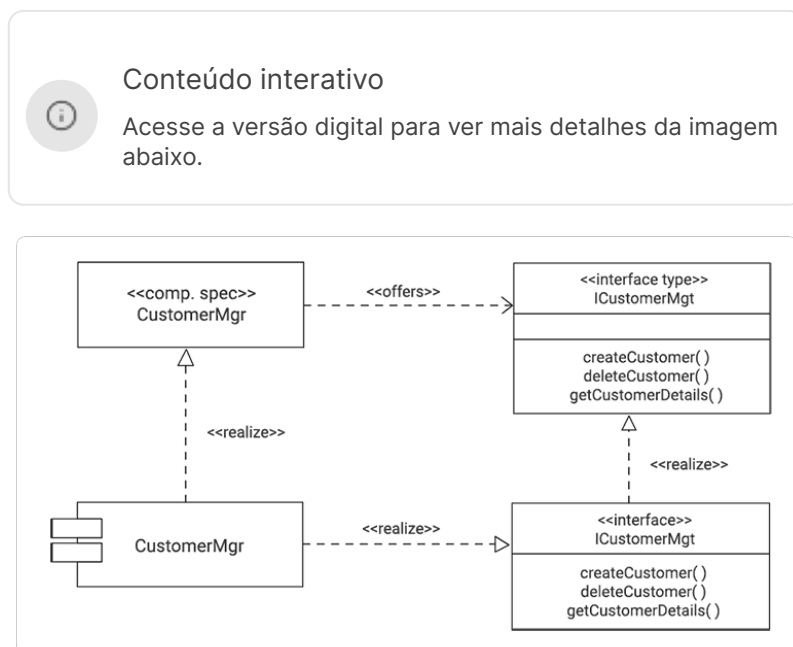
Especificação de Componentes x Construção de Componentes

Em UML, um componente realiza uma ou mais interfaces. Como já discutimos antes, isso é suficiente para modelar alguns dos detalhes da tecnologia de implementação de componentes.

Para lidar com especificação, nós adicionamos alguns estereótipos UML, como especificação de componentes, as classes e suas interfaces.

Uma especificação de componente oferece um ou mais tipos de interfaces, por isso há uma correspondência bastante simples entre os elementos de especificação e os elementos de execução.

UML também define a relação entre o componente e uma interface através de relacionamentos.



Mapeamento de construção e restrições

Os principais problemas com mapeamento de construção e restrições com componentes de tecnologia surgem a partir das seguintes considerações:

- Tipo de parâmetro operação, tipo (in/out/inout/volta) e de referência das restrições. Mecanismos de manipulação de exceção e de erro.
- Restrições heranças de interface e suporte.

- Sequências de operação.
- Propriedades de interface.
- Mecanismos de criação do objeto.
- Disponibilizar eventos.

Heranças de interface e suporte de interfaces

Há uma diferença enorme entre o COM + e EJB, quando se trata de herança de interface e suporte de interfaces.

COM +

COM + permite apenas herança única de interface.

Para permitir que objetos tenham múltiplas classificações, os componentes devem suportar múltiplas interfaces.

Por exemplo, um objeto documento **COM +**, suportando IDocument, IInvoice e IAccStmnt, pode ser:

- Um documento;
- Uma fatura;
- Um extrato de conta.

Além disso, todas as interfaces **COM +** devem, finalmente, herdar de IUnknown-a, interface base **COM +**.

EJB

EJB permite herança múltipla de interfaces e permite que classes Java apoiem múltiplas interfaces, limitando apenas unicamente herança de classe.

Quando registramos uma classe Java como um **EJB** com um ambiente de componentes **EJB**, ficamos restritos à nomeação de uma interface (a chamada interface remota).

O container para o seu **EJB** fornece uma classe de suporte para essa interface, que são, então, delegados a sua classe Java.

Apesar de sua classe Java poder suportar muitas interfaces, o container **EJB** fica limitado a somente um deles.

Isso significa que, se quiser que seu componente suporte múltiplas interfaces, você vai precisar usar herança de interface múltipla para herdar toda a funcionalidade do componente de uma interface pai, que pode ser registrada no ambiente **EJB**.

Essa superinterface é utilizada efetivamente durante a especificação do componente.



Atenção

Devemos tomar cuidado ao usar a herança com tipos de interface. Sabendo-se que esses são artefatos de especificação, de definição e de herança, eles vão se comportar de formas diferentes. Ao contrário de herança de classe de implementação (onde métodos de subclasse podem substituir os métodos da superclasse), as especificações de operação de subtipos só podem incrementar as definições por seus supertipos. Mais especificamente, isso significa que uma especialização de operações pode enfraquecer uma condição prévia (torná-la menos rigorosa) e fortalecer a pós-condição (torná-la mais rigorosa). Isso assegura que qualquer cliente, dependente dos pressupostos e garantias do supertipo, não vai encontrar o subtipo se comportando de forma inesperada.

Propriedades de interfaces

No COM +, uma propriedade de interface é a especificação abreviada para um get e um set, como um par de operações.

Ao usarmos UML para especificar interfaces, definimos que um atributo de interface existe puramente por apoiar a especificação de pré e pós-condições de operações, e você deve, explicitamente, definir quais operações deseja. Por exemplo, a interface base COM + IUnknown tem duas operações para o gerenciamento de contagens de referência:



Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.

AddRef()

Release()

A especificação dessas operações tem de se referir ao atributo de interface “contagem de referência”. Porém, no COM +, definir tal atributo significa, de forma indireta, definir a existência de operações de acesso. Ambos os processos estão diretamente relacionados, o que pode nos trazer problemas.



Saiba mais

Assim, para mapeamento COM +, precisamos de uma maneira para determinar quais as operações devemos mapear para a propriedade COM + taquigrafia. Uma maneira simples é usar uma convenção de nomenclatura. Alternativamente, poderíamos fazer um trabalho mais elaborado e estender a especificação da operação com um valor pré-definido para o nome da propriedade, e interpretar, a partir da existência dentro dos parâmetros, para determinar qual foi o get e o conjunto de informações adquiridas.

Criação de objetos

Tanto no EJB como no COM + usamos uma abordagem de fábrica de objetos, onde um objeto componente é utilizado para criar instâncias de outro componente.

No EJB, a fábrica é o objeto inicial. No COM +, é objeto IClassFactory.

Dentro dos diagramas de arquitetura de componentes e interações de componentes, achamos mais conveniente para omitir esse nível de abstração, pois simplesmente mostram <<criar>> dependências diretamente para o próprio componente.

Quando mapeamos esses objetos para o ambiente de componentes, esses elementos precisam ser adicionados, para que seus diagramas de interação no nível de aplicação mostrem isso.

No COM +, há muita flexibilidade sobre o qual objeto é a fábrica. Isso é feito simplesmente designando um componente que disponibiliza o IClassFactory relevante para uma dada classe.

Esse mesmo elemento também pode fazer outras coisas, e, neste caso, você deve disponibilizar uma funcionalidade para o mesmo.



Atenção

Para beans de sessão EJB, você precisa apenas fornecer uma especificação da interface inicial, mas, para beans de entidade, você precisa fornecer a implementação também.

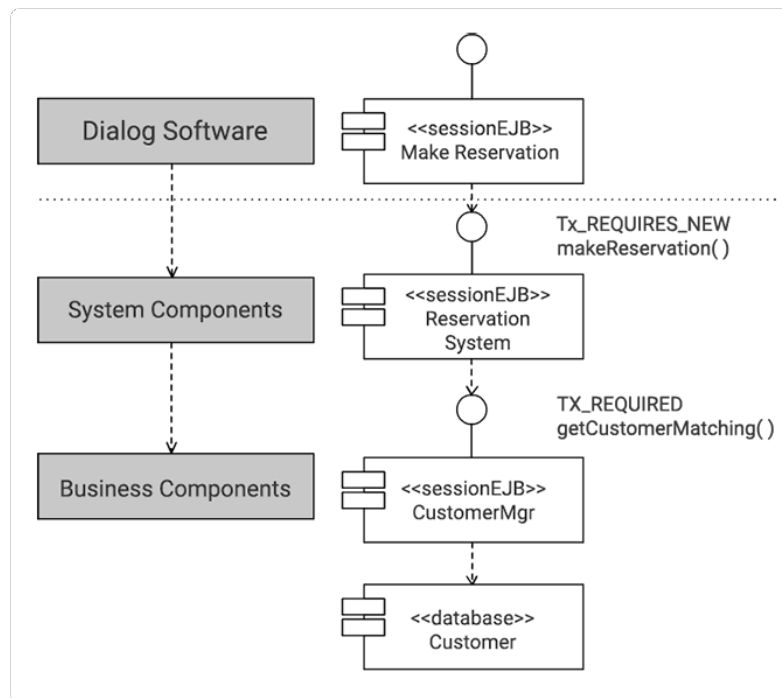
Correspondências da arquitetura da aplicação

Apresentamos aqui uma série de camadas distintas na arquitetura de aplicação, onde cada camada contém diferentes responsabilidades.

Vamos agora indicar brevemente como certas características de especificação dessas camadas podem ser mapeadas para as tecnologias-alvo.

É evidente que as considerações detalhadas de uma arquitetura de sistemas por componentes são muito extensas. Por isso, nos limitamos a algumas observações gerais sobre o uso de sessão e de entidade **EJBs** (essa distinção não existe no **Com +**) e quais as propriedades de transação devem ser utilizadas.

A camada de software de interação é responsável pela gestão da interface com o usuário. Nesse caso, não precisaríamos dividir em componentes, bastaria modelar a relação do usuário com o sistema.



Subcomponentes

Uma especificação é um componente que encapsula a sua funcionalidade (sob a forma de uma ou mais interfaces) que correspondem a uma única unidade de execução.

Isso define a unidade de fornecimento e substituição na arquitetura de aplicativos, geralmente provisionando e substituindo requisitos de granularidade para uma aplicação de negócios de grande escala.



No entanto, às vezes, queremos apresentar um modelo de objetos mais rico e detalhado ao cliente, sem a introdução de um enorme conjunto de componentes separados, cada um agindo de forma isolada.

Como podemos, então, equilibrar a necessidade de definir um componente de forma mais grosseira para efeitos de provisionamento e de forma mais detalhada para a especificação e uso?

Para resolver esse problema, podemos utilizar as especificações de componentes, como subcomponentes (usando um `<<subcomp>>` na especificação do estereótipo), e relacioná-las a um contexto de especificação de componente normal.

A diferença é que esses subcomponentes não têm encapsulamento limite e não representam unidades substituíveis separadamente. Eles expõem uma programação mais limpa do modelo para o código do cliente, permitindo a eficiência na sua aplicação, que é feita através da partilha de detalhes de implementação entre os subcomponentes do mesmo componente.

Os componentes CustomerMgr e clientes devem ser substituídos como uma unidade única.

Outro ponto importante para utilizarmos uma abordagem de subcomponente é quando temos componentes em sua arquitetura, para os quais existiam a intenção de desenvolver, substituir e gerenciar separadamente, mas a interação entre eles é tal que decidimos pelas vantagens de uma única implementação de desempenho.

A abordagem subcomponente permite-lhe manter a especificação conforme sua estruturação de negócios e detalhamento para implementação.

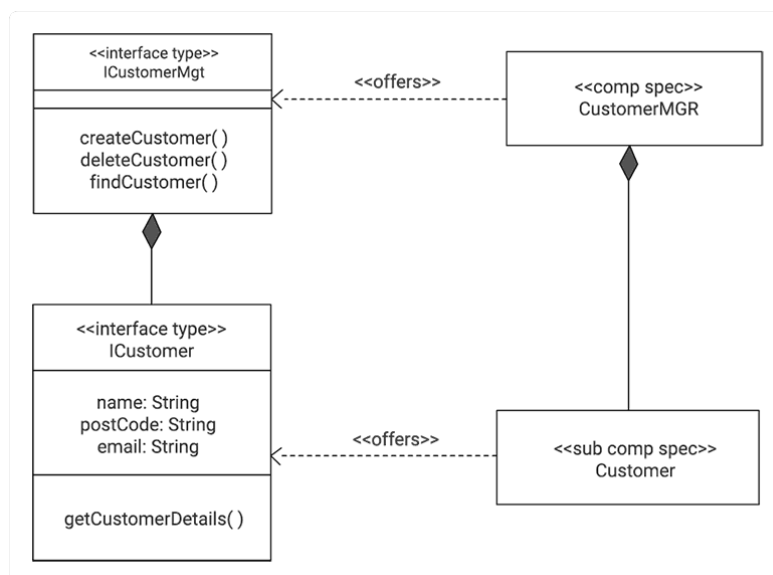
Instâncias são criadas por operações específicas

Subcomponentes não têm objetos de fábrica explícitas como os componentes. Em vez disso, suas instâncias são criadas por operações específicas do componente pai ou irmão.

Por exemplo, a operação `ICustomerMgt :: createCustomer ()` iria criar uma instância do subcomponente do cliente.

Subcomponente objeto estado share com o seu componente, passando a existir somente em tempo de execução.

Um componente e todos os seus subcomponentes devem ser embalados em conjunto para o mesmo módulo (.exe, .dll .jar).



Interação com sistemas existentes

Em alguns momentos de nossa disciplina, tratamos do desenvolvimento somente utilizando o conceito de novos componentes. Porém, no mundo real, quando utilizamos a arquitetura de sistemas por componentes, a maioria dos componentes já estão prontos em sistemas existentes ou em aplicativos de prateleira.

Para os casos de utilização de componentes prontos, devemos fazê-lo através de APIs, disponibilizadas por esses componentes.



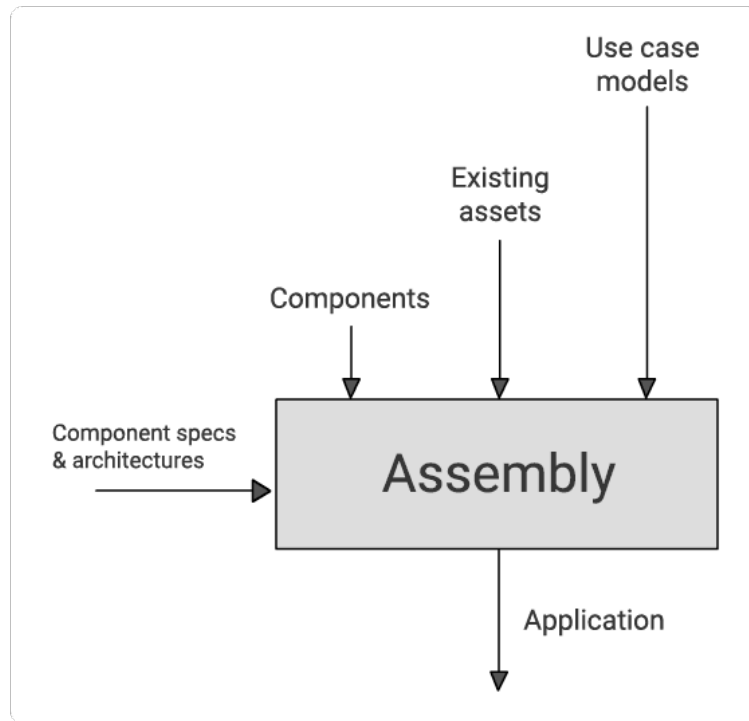
Todos os elementos de interação com os componentes prontos devem ser feitos através de interfaces, que precisam ser bem especificadas e definidas para não deixar dúvidas de seus usos e de seus resultados.

Construção

A construção é o processo de reunir componentes de software existentes ativos em um ambiente de TI e projetar uma interface de usuário para esse sistema, com base nos modelos para formar uma aplicação.

A construção compartilha várias das características presentes na configuração padrão do ambiente, principalmente no que se refere a práticas de gerenciamento.

Cada componente individual disponível pode ser visto como um item de configuração separado a ser utilizado, mantendo-se um controle de versão.



Verificando o aprendizado

Sobre “projetar componentes”, assinale a alternativa cujo o projeto NÃO faz parte deste passo:

☐ A Projeto do modelo de classes.

☒ B Projeto do modelo de casos de uso.

☐ C Projeto do modelo de dados.

☐ D Projeto do modelo de componentes.



A alternativa B está correta.

O projeto do modelo de casos de uso faz parte da modelagem de requisitos, não do passo de projetar componentes.

Considerações finais

Neste conteúdo, abordamos os principais desafios relacionados à transformação de requisitos em especificações técnicas claras, destacando a importância dessa etapa para o sucesso no desenvolvimento de sistemas. Iniciamos com a análise do papel da especificação de componentes como elemento de ligação entre as necessidades das partes interessadas e a implementação efetiva pelos programadores. Evidenciamos como uma especificação bem elaborada contribui para a construção de soluções alinhadas às expectativas dos usuários e aos padrões da arquitetura de sistemas.

Em seguida, exploramos as ações necessárias para que a especificação represente de forma consistente os requisitos levantados, enfatizando sua função como tradução entre linguagem de negócios e linguagem técnica. Essa abordagem permitiu compreender a especificação de componentes não apenas como um registro técnico, mas como uma etapa estratégica para o desenvolvimento de software eficiente e sustentável.

Avançamos para o último estágio da metodologia apresentada: a construção de componentes. Detalhamos o processo de construção com base nos elementos definidos pelo arquiteto de sistemas, considerando boas práticas e padrões técnicos, essenciais para garantir a qualidade do produto final. Também discutimos a necessidade de manter as especificações o mais independentes possível das tecnologias utilizadas, reconhecendo os limites de compatibilidade e a necessidade de interpretação em alguns casos.

Com isso, reforçamos a relevância das etapas de especificação e construção de componentes como fundamentos indispensáveis para projetos bem-sucedidos, integrando teoria, prática e tomada de decisão técnica no contexto da arquitetura de sistemas.

Explore +

Para saber mais sobre os assuntos estudados leia:

- O artigo "[Arquitetura de Sistemas de Informação - Evolução e Análise Comparativa de Modelos](#)".
- O material [Requisitos de Usuário e Requisitos do Sistema](#).

Acesse:

- O site da School of Computer Science e busque pelo termo "Software Architecture".
- Acesse o site DEVMEDIA e veja a "Metodologia de desenvolvimento de software".

Referências

- BASS, Len; CLEMENTS, Paul. **Software Architecture in Practice**. 2. ed. Nova Jersey: Pearson, 2003.

- CHESSMAN, John Daniels. **UML Components** – A Simple Process For Specifyng Component -Based Software. Boston: Addison-Wesley, 2000.
- PRADO LEITE, Julio Cesar Sampaio. **Engenharia de Requisitos**. Consultado na internet em 4 mai. 2016.
- RATIONAL SOFTWARE CORP. **Conceitos**: Tipos de Requisitos. Consultado na internet em 4 mai. 2016.
- SILVEIRA, Guilherme. **Introdução à arquitetura de design de software**. Rio de Janeiro: Elsevier (Livros Digitais), 2015.
- TERUEL, Evandro Carlos. **Arquitetura de sistemas para Web com Java Utilizando Design Patterns e Frameworks**. Rio de Janeiro: Ciência Moderna, 2012.